

JavaScript

INTRODUÇÃO A LINGUAGEM DE PROGRAMAÇÃO JAVASCRIPT
WANDERLEI DE PONTE DOS SANTOS

Cronograma

Capítulo 1: Introdução ao JavaScript e Configuração do Ambiente

◆ Teoria:

- O que é JavaScript?
- História e evolução do JavaScript.
- Onde o JavaScript é usado? (Front-end, Back-end, Mobile, etc.)
- Diferença entre JavaScript e outras linguagens.
- Configuração do ambiente: Navegador e Console do DevTools.
- Introdução ao VS Code (ou outro editor de texto).
- Primeiros passos: `console.log()` e comentários.
- Exercícios de Fixação:
 - Escrever "Olá, Mundo!" no console.
 - Comentar partes do código.
 - Explorar o Console do Navegador.
- Aplicação Prática:
 - Criar uma página HTML simples com um script que exibe uma mensagem no console.

Capítulo 2: Variáveis, Tipos de Dados e Operadores

◆ Teoria:

- O que são variáveis? (`let`, `const`, `var`).
- Tipos de dados primitivos: String, Number, Boolean, Null, Undefined.
- Operadores: Aritméticos, Comparação, Lógicos.
- Conversão de tipos (type coercion).
- Exercícios de Fixação:
 - Declarar variáveis e atribuir valores.
 - Realizar operações matemáticas e comparar valores.
 - Concatenar strings e números.
- Aplicação Prática:
 - Criar uma página que calcula e exibe a soma de dois números inseridos pelo usuário.

Capítulo 3: Estruturas de Controle (Condicionais e Loops)

◆ Teoria:

- Condicionais: `if`, `else`, `else if`, `switch`.
- Operador ternário.
- Loops: `for`, `while`, `do...while`.
- Break e Continue.
- Exercícios de Fixação:
 - Criar condições para verificar números pares/ímpares.
 - Usar loops para iterar sobre arrays ou strings.
 - Implementar um contador simples.
- Aplicação Prática:
 - Criar uma página que verifica se um número é primo e exibe o resultado.

◆ Capítulo 4: Funções

- Teoria:
 - O que são funções e por que usá-las?
- Declaração de funções: `function`, arrow functions.
- Parâmetros e retorno.

- Escopo de variáveis (global e local).
- Funções anônimas e callbacks.
- Exercícios de Fixação:
 - Criar funções para realizar operações matemáticas.
 - Implementar funções que recebem e retornam valores.
 - Usar callbacks simples.
- Aplicação Prática:
 - Criar uma página com uma calculadora simples usando funções.

Capítulo 5: Arrays e Objetos

◆ Teoria:

- O que são arrays? Métodos comuns: ``push``, ``pop``, ``shift``, ``unshift``, ``slice``, ``splice``.
- Iteração em arrays: ``forEach``, ``map``, ``filter``, ``reduce``.
- O que são objetos? Propriedades e métodos.
- Acessar e modificar propriedades de objetos.
- Exercícios de Fixação:
 - Manipular arrays: adicionar, remover e filtrar elementos.
 - Criar objetos e acessar suas propriedades.
 - Usar métodos de iteração em arrays.
- Aplicação Prática:
 - Criar uma página que exibe uma lista de tarefas (to-do list) usando arrays e objetos.

Capítulo 6: Manipulação do DOM (Document Object Model)

◆ Teoria:

- O que é o DOM?
- Selecionar elementos: ``getElementById``, ``querySelector``, ``querySelectorAll``.
- Manipular elementos: alterar conteúdo, estilos e atributos.
- Eventos: ``addEventListener``, tipos de eventos (click, mouseover, etc.).
- Exercícios de Fixação:
 - Alterar o conteúdo de uma página dinamicamente.
 - Adicionar eventos a botões e outros elementos.
 - Criar interações simples com o usuário.
- Aplicação Prática:
 - Criar uma página com um botão que altera o texto e a cor de um elemento ao ser clicado.

Capítulo 7: Validação de Formulários e Interação com o Usuário

◆ Teoria:

- Capturar dados de formulários.
- Validação de campos (required, length, regex).
- Exibir mensagens de erro/sucesso.
- Prevenção do comportamento padrão (``preventDefault``).
- Exercícios de Fixação:
 - Validar campos de um formulário.
 - Exibir mensagens personalizadas com base na entrada do usuário.
- Aplicação Prática:
 - Criar um formulário de contato com validação em tempo real.

Capítulo 8: Armazenamento no Navegador (LocalStorage e sessionStorage)

◆ Teoria:

- Diferença entre LocalStorage e sessionStorage.
- Como salvar e recuperar dados.

- Limitações e boas práticas.
- Exercícios de Fixação:
 - Salvar e recuperar dados do LocalStorage.
 - Criar um sistema de preferências do usuário.
- Aplicação Prática:
 - Criar uma página que salva o tema preferido do usuário (claro/escuro).

Capítulo 9: Introdução ao AJAX e Fetch API

◆ Teoria:

- O que é AJAX?
- Requisições assíncronas com Fetch API.
- Trabalhando com JSON.
- Promises e async/await.
- Exercícios de Fixação:
 - Fazer requisições a APIs públicas.
 - Exibir dados dinamicamente na página.
- Aplicação Prática:
 - Criar uma página que consome uma API de previsão do tempo e exibe os dados.

Capítulo 10: Programação Orientada a Objetos (POO) em JavaScript

◆ Teoria:

- O que é POO?
- Classes e objetos.
- Herança, encapsulamento e polimorfismo.
- Métodos estáticos e instâncias.
- Exercícios de Fixação:
 - Criar classes e instâncias.
 - Implementar herança entre classes.
- Aplicação Prática:
 - Criar um sistema simples de gerenciamento de produtos usando POO.

Capítulo 11: Introdução a Frameworks (React, Vue ou Angular)

◆ Teoria:

- O que são frameworks e por que usá-los?
- Comparação entre React, Vue e Angular.
- Componentes e reatividade.
- Exercícios de Fixação:
 - Criar componentes simples.
 - Manipular estado e props.
- Aplicação Prática:
 - Criar uma aplicação simples com um framework escolhido.

Capítulo 12: Projeto Final

◆ Teoria:

- Revisão de todos os conceitos aprendidos.
- Boas práticas de desenvolvimento.
- Introdução ao versionamento com Git.
- Aplicação Prática:
 - Desenvolver um projeto completo (ex: blog, e-commerce, dashboard).
 - Publicar o projeto no GitHub Pages ou outro serviço de hospedagem.

Capítulo 1:

Introdução ao JavaScript e Configuração do Ambiente

1. Teoria: Introdução ao JavaScript

O que é JavaScript?

JavaScript é uma linguagem de programação usada principalmente para adicionar interatividade a páginas da web. Ele permite que os desenvolvedores criem conteúdos dinâmicos, controlem multimídia, animem elementos e muito mais. Diferente de HTML (que estrutura o conteúdo) e CSS (que estiliza o conteúdo), o JavaScript torna as páginas web interativas e funcionais.

História e Evolução do JavaScript:

- Criado por Brendan Eich em 1995 para o navegador Netscape.
- Inicialmente chamado de Mocha, depois LiveScript, e finalmente JavaScript.
- Hoje, é uma das linguagens mais populares do mundo, usada tanto no front-end quanto no back-end (com Node.js).

Onde o JavaScript é usado?

- Front-end: Interatividade em páginas web (ex: validação de formulários, animações).
- Back-end: Desenvolvimento de servidores com Node.js.
- Mobile: Desenvolvimento de aplicativos com frameworks como React Native.
- Desktop: Aplicações desktop com Electron.

Diferença entre JavaScript e outras linguagens:

- JavaScript é interpretada (executada diretamente no navegador), enquanto linguagens como Java ou C++ são compiladas.
- É uma linguagem de alto nível, ou seja, mais próxima da linguagem humana do que do código de máquina.

2. Configuração do Ambiente

Ferramentas Necessárias:

1. Navegador: Chrome, Firefox, Edge, etc. (recomendamos o Chrome por suas ferramentas de desenvolvedor).
2. Editor de Texto: VS Code (gratuito e popular) ou qualquer outro de sua preferência.

Passo a Passo:

1. Abrir o Console do Navegador:
 - No Chrome: Clique com o botão direito na página → "Inspecionar" → Abra a aba "Console".
 - Ou pressione "Ctrl + Shift + J" (Windows/Linux) ou "Cmd + Option + J" (Mac).
2. Instalar o VS Code:
 - Baixe e instale o VS Code em code.visualstudio.com.
 - Extensões recomendadas: Live Server (para visualizar páginas web em tempo real).

3. Primeiros Passos: "console.log()" e Comentários

O que é "console.log()"?

- É uma função usada para exibir mensagens no console do navegador.
- Útil para depuração (encontrar erros) e para testar pequenos trechos de código.

Sintaxe:

```
console.log("Olá, Mundo!");
```

Comentários:

- Comentários são trechos de código ignorados pelo navegador.
- Úteis para explicar o código ou desativar partes temporariamente.
- Comentário de uma linha: // Este é um comentário.

- Comentário de múltiplas linhas:

```
/*  
    Este é um comentário  
    de múltiplas linhas.  
*/
```

4. Exercícios de Fixação

1. Escreva "Olá, Mundo!" no console:
 - Abra o console do navegador e digite:
`console.log("Olá, Mundo!");`
2. Use comentários:
 - Adicione um comentário de uma linha antes do `console.log``.
 - Adicione um comentário de múltiplas linhas explicando o que o código faz.
3. Explore o Console:
 - Digite algumas operações matemáticas no console, como:
`console.log(2 + 3);`
`console.log(10 / 2);`

5. Aplicação Prática: Criando uma Página HTML com JavaScript

Passo a Passo:

1. Crie uma pasta no seu computador chamada `projeto-js``.
2. Dentro dessa pasta, crie um arquivo chamado `index.html``.
3. Abra o arquivo no VS Code e adicione o seguinte código:

```
<!DOCTYPE html>  
<html lang="pt-BR">  
  <head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>Meu Primeiro Projeto JavaScript</title>  
  </head>  
  <body>  
    <h1>Olá, Mundo!</h1>  
    <script>  
      // Exibe uma mensagem no console  
      console.log("Esta é a minha primeira página com JavaScript!");  
    </script>  
  </body>  
</html>
```

4. Salve o arquivo e abra-o no navegador.
5. Abra o console do navegador e veja a mensagem exibida.

6. Desafio

- Modifique o arquivo `index.html`` para exibir uma segunda mensagem no console, como:
`console.log("Estou aprendendo JavaScript!");`

Capítulo 2:

Variáveis, Tipos de Dados e Operadores

1. Teoria: Variáveis

O que são variáveis?

Variáveis são como "caixas" que armazenam dados. Elas permitem que você guarde valores (como números, textos, etc.) e os use posteriormente no seu código.

Declarando Variáveis:

Em JavaScript, usamos três palavras-chave para declarar variáveis:

1. **let**: Declara uma variável que pode ser alterada posteriormente.

```
let nome = "João";  
nome = "Maria"; // Valor alterado
```

2. **const**: Declara uma variável que **não pode** ser alterada (constante).

```
const PI = 3.14;  
// PI = 3.15; // Isso causaria um erro
```

3. **var**: Forma antiga de declarar variáveis (não recomendada hoje em dia).

Boas Práticas:

- Use “const” por padrão e “let” apenas quando o valor precisar mudar.
- Evite “var” para evitar problemas de escopo.

2. Teoria: Tipos de Dados

JavaScript é uma linguagem de **tipagem dinâmica**, ou seja, uma variável pode armazenar diferentes tipos de dados ao longo do tempo.

Tipos de Dados Primitivos:

1. **String**: Texto (entre aspas simples ou duplas).

```
let nome = "Ana";  
let mensagem = 'Olá, mundo!';
```

2. **Number**: Números (inteiros ou decimais).

```
let idade = 25;  
let preco = 19.99;
```

3. **Boolean**: Verdadeiro (``true``) ou falso (``false``).

```
let isLoggedIn = true;  
let isAdmin = false;
```

4. **Null**: Representa um valor nulo ou vazio.

```
let endereco = null;
```

5. **Undefined**: Representa uma variável que ainda não foi definida.

```
let telefone;  
console.log(telefone); // undefined
```

Verificando o Tipo de Dado:

Use o operador “typeof” para descobrir o tipo de uma variável.

```
console.log(typeof "Texto"); // "string"
console.log(typeof 42); // "number"
console.log(typeof true); // "boolean"
```

3. Teoria: Operadores

Operadores Aritméticos:

- `+` (adição), `-` (subtração), `\*` (multiplicação), `/` (divisão), `%` (resto da divisão).

```
let soma = 10 + 5; // 15
let subtracao = 10 - 5; // 5
let multiplicacao = 10 * 2; // 20
let divisao = 10 / 2; // 5
let resto = 10 % 3; // 1
```

Operadores de Comparação:

- `==` (igual a), `===` (igual a e do mesmo tipo), `!=` (diferente de), `!==` (diferente de e do mesmo tipo),
`>` (maior que), `<` (menor que), `>=` (maior ou igual), `<=` (menor ou igual).

```
console.log(10 == "10"); // true (valor igual)
console.log(10 === "10"); // false (valor e tipo diferentes)
console.log(10 > 5); // true
```

Operadores Lógicos:

- `&&` (E), `||` (OU), `!` (NÃO).

```
console.log(true && false); // false
console.log(true || false); // true
console.log(!true); // false
```

Operadores de Atribuição:

- `=` (atribuição), `+=` (incremento), `-=` (decremento), etc.

```
let x = 10;
x += 5; // x = x + 5 → 15
```

4. Exercícios de Fixação

1. Declare variáveis:

- Crie uma variável `nome` e atribua seu nome a ela.
- Crie uma variável `idade` e atribua sua idade.
- Crie uma constante `PI` com o valor 3.14.

2. Operações matemáticas:

- Crie duas variáveis `num1` e `num2` com valores numéricos.
- Calcule a soma, subtração, multiplicação e divisão entre elas.

3. Comparações:

- Compare se `num1` é maior que `num2`.
- Verifique se `num1` é igual a `num2` em valor e tipo.

4. Operadores lógicos:

- Crie duas variáveis booleanas `temCarteira` e `temCarro`.
- Verifique se a pessoa pode dirigir (tem carteira E tem carro).

5. Aplicação Prática: Calculadora Simples

Passo a Passo:

1. Crie um arquivo `calculadora.html` com o seguinte código:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Calculadora Simples</title>
</head>
<body>
  <h1>Calculadora Simples</h1>
  <script>
    // Declare duas variáveis com números
    let num1 = 10;
    let num2 = 5;

    // Realize operações
    let soma = num1 + num2;
    let subtracao = num1 - num2;
    let multiplicacao = num1 * num2;
    let divisao = num1 / num2;

    // Exiba os resultados no console
    console.log("Soma:", soma);
    console.log("Subtração:", subtracao);
    console.log("Multiplicação:", multiplicacao);
    console.log("Divisão:", divisao);
  </script>
</body>
</html>
```

2. Abra o arquivo no navegador e veja os resultados no console.

6. Desafio Opcional

- Modifique a calculadora para pedir os valores de `num1` e `num2` ao usuário usando `prompt()`.
- Exiba os resultados diretamente na página usando `document.write()` ou manipulando o DOM (se já souber como).

Capítulo 3:

Estruturas de Controle (Condicionais e Loops)

1. Teoria: Condicionais

O que são condicionais?

Condicionais permitem que você execute diferentes blocos de código com base em condições específicas. Elas são essenciais para tomar decisões em seu programa.

Estruturas Condicionais:

1. if: Executa um bloco de código se a condição for verdadeira.

```
let idade = 18;
if (idade >= 18) {
    console.log("Você é maior de idade.");
}
```

2. else: Executa um bloco de código se a condição do `if` for falsa.

```
if (idade >= 18) {
    console.log("Você é maior de idade.");
} else {
    console.log("Você é menor de idade.");
}
```

3. else if: Permite testar múltiplas condições.

```
let nota = 7;
if (nota >= 9) {
    console.log("Excelente!");
} else if (nota >= 7) {
    console.log("Bom!");
} else {
    console.log("Precisa melhorar.");
}
```

4. Operador Ternário:

- Forma abreviada de escrever uma condição `if...else`.
- ```
let status = (idade >= 18) ? "Maior de idade" : "Menor de idade";
console.log(status);
```

5. switch: Útil para comparar uma variável com múltiplos valores.

```
let dia = 3;
switch (dia) {
 case 1:
 console.log("Domingo");
 break;
 case 2:
 console.log("Segunda-feira");
 break;
 case 3:
 console.log("Terça-feira");
 break;
 default:
 console.log("Dia inválido");
}
```

## 2. Teoria: Loops

O que são loops?

Loops permitem que você execute um bloco de código repetidamente enquanto uma condição for verdadeira. Eles são úteis para iterar sobre arrays, processar dados repetitivos, etc.

Tipos de Loops:

1. for: Executa um bloco de código um número específico de vezes.

```
for (let i = 0; i < 5; i++) {
 console.log("Número:", i);
}
```

2. while: Executa um bloco de código enquanto a condição for verdadeira.

```
let contador = 0;
while (contador < 5) {
 console.log("Contador:", contador);
 contador++;
}
```

3. do...while: Semelhante ao **“while”**, mas o bloco de código é executado pelo menos uma vez, mesmo que a condição seja falsa.

```
let contador = 0;
do {
 console.log("Contador:", contador);
 contador++;
} while (contador < 5);
```

4. break e continue:

- break: Interrompe a execução do loop.
- continue: Pula para a próxima iteração do loop.

```
for (let i = 0; i < 10; i++) {
 if (i === 5) {
 break; // Interrompe o loop quando i for 5
 }
 console.log(i);
}
```

## 3. Exercícios de Fixação

1. Condicionais:

- Crie uma variável `idade` e verifique se a pessoa pode votar ( $idade \geq 16$ ).
- Use `else if` para classificar uma nota em "A", "B", "C", "D" ou "F".

2. Loops:

- Use um loop `for` para exibir os números de 1 a 10.
- Use um loop `while` para exibir os números pares de 0 a 20.
- Use `break` para interromper um loop quando encontrar o número 5.

### 3. Desafio:

- Crie um loop que exiba a tabuada de um número (ex: tabuada do 7).

### 4. Aplicação Prática: Verificador de Números Primos

#### Passo a Passo:

1. Crie um arquivo “**primos.html**” com o seguinte código:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Verificador de Números Primos</title>
</head>
<body>
 <h1>Verificador de Números Primos</h1>
 <script>
 let numero = 29; // Número a ser verificado
 let isPrimo = true;

 // Verifica se o número é primo
 for (let i = 2; i < numero; i++) {
 if (numero % i === 0) {
 isPrimo = false;
 break;
 }
 }

 // Exibe o resultado
 if (isPrimo) {
 console.log(numero + " é um número primo.");
 } else {
 console.log(numero + " não é um número primo.");
 }
 </script>
</body>
</html>
```

2. Abra o arquivo no navegador e veja o resultado no console.

### 5. Desafio

- Modifique o verificador de números primos para pedir ao usuário um número usando `prompt()`.
- Exiba o resultado diretamente na página usando `document.write()` ou manipulando o DOM.

## Capítulo 4:

### Funções

## 1. Teoria: O que são Funções?

### Definição:

Funções são blocos de código que realizam uma tarefa específica. Elas permitem que você organize seu código, evite repetições e reutilize lógica em diferentes partes do programa.

### Por que usar funções?

- Reutilização de código: Escreva uma vez, use várias vezes.
- Organização: Divida o código em partes menores e mais gerenciáveis.
- Facilidade de manutenção: Corrija ou atualize o código em um único lugar.

## 2. Declarando Funções

### Sintaxe Básica:

```
function nomeDaFuncao(parametro1, parametro2) {
 // Código a ser executado
 return resultado; // Opcional
}
```

### Exemplo:

```
function saudacao(nome) {
 return "Olá, " + nome + "!";
}

console.log(saudacao("João")); // "Olá, João!"
```

### Funções Anônimas:

Funções sem nome, geralmente atribuídas a variáveis ou usadas como callbacks.

```
const soma = function(a, b) {
 return a + b;
};

console.log(soma(2, 3)); // 5
```

### Arrow Functions (Funções de Seta):

Sintaxe mais curta e moderna para funções.

```
const multiplicacao = (a, b) => a * b;
console.log(multiplicacao(2, 3)); // 6
```

## 3. Parâmetros e Retorno

### Parâmetros:

- São valores que você passa para a função.
- Podem ser usados dentro da função como variáveis.

### Retorno:

- A função pode retornar um valor usando a palavra-chave `return`.
- Se não houver `return`, a função retorna `undefined`.

Exemplo:

```
function calcularArea(raio) {
 return Math.PI * raio * raio;
}

let area = calcularArea(5);
console.log("Área do círculo:", area);
```

#### 4. Escopo de Variáveis

Escopo Global:

- Variáveis declaradas fora de funções são acessíveis em todo o código.

Escopo Local:

- Variáveis declaradas dentro de funções são acessíveis apenas dentro delas.

Exemplo:

```
let global = "Eu sou global";

function testeEscopo() {
 let local = "Eu sou local";
 console.log(global); // Acessível
 console.log(local); // Acessível
}

testeEscopo();
console.log(global); // Acessível
console.log(local); // Erro: local não está definido
```

#### 5. Exercícios de Fixação

1. Crie uma função:

- Escreva uma função `soma` que recebe dois números e retorna a soma deles.

2. Função com condicional:

- Crie uma função `maiorNumero` que recebe dois números e retorna o maior deles.

3. Arrow Function:

- Converta a função `soma` para uma arrow function.

4. Escopo:

- Declare uma variável global e outra local dentro de uma função. Tente acessá-las fora da função.

#### 6. Aplicação Prática: Calculadora com Funções

Passo a Passo:

1. Crie um arquivo **“calculadora-funcoes.html”** com o seguinte código:

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Calculadora com Funções</title>
</head>
<body>
 <h1>Calculadora com Funções</h1>
 <script>
 // Funções da calculadora
 function soma(a, b) {
 return a + b;
 }

 function subtracao(a, b) {
 return a - b;
 }

 function multiplicacao(a, b) {
 return a * b;
 }

 function divisao(a, b) {
 if (b === 0) {
 return "Erro: divisão por zero!";
 }
 return a / b;
 }

 // Testando as funções
 console.log("Soma:", soma(10, 5));
 console.log("Subtração:", subtracao(10, 5));
 console.log("Multiplicação:", multiplicacao(10, 5));
 console.log("Divisão:", divisao(10, 5));
 </script>
</body>
</html>

```

2. Abra o arquivo no navegador e veja os resultados no console.

## 7. Desafio

- Adicione uma função `potencia` que recebe dois números (base e expoente) e retorna o resultado.
- Modifique o código para pedir ao usuário os números e a operação desejada usando `prompt()`.
- Exiba o resultado diretamente na página.

## Capítulo 5:

### Arrays e Objetos

## 1. Teoria: Arrays

### O que são Arrays?

Arrays são estruturas que permitem armazenar múltiplos valores em uma única variável. Eles são úteis para trabalhar com listas, coleções de dados, etc.

### Sintaxe:

```
let frutas = ["Maçã", "Banana", "Laranja"];
```

### Acessando Elementos:

- Os elementos de um array são acessados por índices (começando em 0).

```
console.log(frutas[0]); // "Maçã"
console.log(frutas[1]); // "Banana"
```

### Métodos Comuns:

1. `push()`: Adiciona um elemento ao final do array.

```
frutas.push("Uva");
console.log(frutas); // ["Maçã", "Banana", "Laranja", "Uva"]
```

2. `pop()`: Remove o último elemento do array.

```
frutas.pop();
console.log(frutas); // ["Maçã", "Banana", "Laranja"]
```

3. `length`: Retorna o número de elementos no array.

```
console.log(frutas.length); // 3
```

4. `forEach()`: Itera sobre cada elemento do array.

```
frutas.forEach(function(fruta) {
 console.log(fruta);
});
```

5. `map()`: Cria um novo array com os resultados de uma função aplicada a cada elemento.

```
let frutasMaiusculas = frutas.map(fruta => fruta.toUpperCase());
console.log(frutasMaiusculas); // ["MAÇÃ", "BANANA", "LARANJA"]
```

6. `filter()`: Cria um novo array com elementos que passam em um teste.

```
let frutasComA = frutas.filter(fruta => fruta.includes("a"));
console.log(frutasComA); // ["Banana", "Laranja"]
```

## 2. Teoria: Objetos

### O que são Objetos?

Objetos são estruturas que armazenam dados em pares de chave-valor. Eles são úteis para representar entidades do mundo real, como usuários, produtos, etc.

### Sintaxe:

```
let usuario = {
 nome: "João",
 idade: 25,
 isAdmin: false
}
```

```
};
```

Acessando Propriedades:

```
console.log(usuario.nome); // "João"
console.log(usuario["idade"]); // 25
```

Adicionando/Modificando Propriedades:

```
usuario.email = "joao@example.com";
usuario.idade = 26;
```

Métodos:

- Funções dentro de objetos são chamadas de métodos.

```
let calculadora = {
 soma: function(a, b) {
 return a + b;
 },
 subtracao: function(a, b) {
 return a - b;
 }
};

console.log(calculadora.soma(10, 5)); // 15
```

### 3. Exercícios de Fixação

Arrays:

1. Crie um array `numeros` com os valores [10, 20, 30, 40, 50].

- Use `push()` para adicionar o número 60.
- Use `pop()` para remover o último número.
- Use `forEach()` para exibir cada número no console.

2. Crie um array `nomes` com os valores ["Ana", "Carlos", "Bianca", "Daniel"].

- Use `map()` para criar um novo array com todos os nomes em maiúsculas.
- Use `filter()` para criar um novo array com nomes que começam com a letra "B".

3. Crie uma função `somaArray` que recebe um array de números e retorna a soma deles.

- Teste com o array [1, 2, 3, 4, 5].

Objetos:

4. Crie um objeto `carro` com as propriedades `marca`, `modelo` e `ano`.

- Adicione um método `detalhes` que retorna uma string com todas as informações do carro.

5. Crie um objeto `aluno` com as propriedades `nome`, `notas` (um array de notas) e um método `media` que calcula a média das notas.

- Teste com o aluno "Maria" e as notas [7, 8, 9].

### 4. Aplicação Prática: Lista de Tarefas (To-Do List)

Passo a Passo:

1. Crie um arquivo `todo-list.html` com o seguinte código:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Lista de Tarefas</title>
</head>
<body>
 <h1>Lista de Tarefas</h1>
 <script>
 let tarefas = [
 { id: 1, descricao: "Fazer compras", concluida: false },
 { id: 2, descricao: "Estudar JavaScript", concluida: true },
 { id: 3, descricao: "Lavar o carro", concluida: false }
];

 // Exibir tarefas não concluídas
 let tarefasPendentes = tarefas.filter(tarefa => !tarefa.concluida);
 console.log("Tarefas Pendentes:", tarefasPendentes);

 // Marcar uma tarefa como concluída
 function concluirTarefa(id) {
 let tarefa = tarefas.find(tarefa => tarefa.id === id);
 if (tarefa) {
 tarefa.concluida = true;
 }
 }

 concluirTarefa(1);
 console.log("Tarefas atualizadas:", tarefas);
 </script>
</body>
</html>
```

2. Abra o arquivo no navegador e veja os resultados no console.

## 5. Respostas dos Exercícios

Arrays:

1.  
let numeros = [10, 20, 30, 40, 50];  
numeros.push(60);  
numeros.pop();  
numeros.forEach(numero => console.log(numero));
2.  
let nomes = ["Ana", "Carlos", "Bianca", "Daniel"];  
let nomesMaiusculos = nomes.map(nome => nome.toUpperCase());

```
let nomesComB = nomes.filter(nome => nome.startsWith("B"));
console.log(nomesMaiusculos); // ["ANA", "CARLOS", "BIANCA", "DANIEL"]
console.log(nomesComB); // ["Bianca"]
```

3.

```
function somaArray(arr) {
 return arr.reduce((total, num) => total + num, 0);
}
console.log(somaArray([1, 2, 3, 4, 5])); // 15
```

Objetos:

4.

```
let carro = {
 marca: "Toyota",
 modelo: "Corolla",
 ano: 2020,
 detalhes: function() {
 return `${this.marca} ${this.modelo} (${this.ano})`;
 }
};
console.log(carro.detalhes()); // "Toyota Corolla (2020)"
```

5.

```
let aluno = {
 nome: "Maria",
 notas: [7, 8, 9],
 media: function() {
 let soma = this.notas.reduce((total, nota) => total + nota, 0);
 return soma / this.notas.length;
 }
};
console.log(aluno.media()); // 8
```

6. Desafio Opcional

- Crie uma função `adicionarTarefa` que recebe uma descrição e adiciona uma nova tarefa à lista de tarefas.
- Crie uma função `removerTarefa` que recebe um ID e remove a tarefa correspondente.

## Capítulo 6:

### Manipulação do DOM (Document Object Model)

## 1. Teoria: O que é o DOM?

### Definição:

O DOM (Document Object Model) é uma representação em árvore de todos os elementos de uma página HTML. Ele permite que o JavaScript acesse, modifique e interaja com o conteúdo, estrutura e estilo da página.

### Estrutura do DOM:

- Cada elemento HTML é representado como um nó no DOM.
- Os nós podem ser elementos, atributos, textos, etc.
- Exemplo:

```
<div id="container">
 <h1>Título</h1>
 <p>Parágrafo</p>
</div>
```

No DOM, isso seria:

```
div (id="container")
├── h1
│ └── "Título"
└── p
 └── "Parágrafo"
```

## 2. Selecionando Elementos

### Métodos de Seleção:

1. `document.getElementById()`: Seleciona um elemento pelo ID.  
`let titulo = document.getElementById("titulo");`
2. `document.querySelector()`: Seleciona o primeiro elemento que corresponde a um seletor CSS.  
`let paragrafo = document.querySelector("p");`
3. `document.querySelectorAll()`: Seleciona todos os elementos que correspondem a um seletor CSS.  
`let itens = document.querySelectorAll(".item");`
4. `document.getElementsByClassName()`: Seleciona elementos pela classe.  
`let botoes = document.getElementsByClassName("botao");`
5. `document.getElementsByTagName()`: Seleciona elementos pela tag.  
`let divs = document.getElementsByTagName("div");`

## 3. Manipulando Elementos

### Alterando Conteúdo:

1. `innerHTML`: Altera o conteúdo HTML de um elemento.  
`titulo.innerHTML = "Novo Título";`
2. `textContent`: Altera o texto de um elemento.  
`paragrafo.textContent = "Este é um novo parágrafo.";`

### Alterando Atributos:

1. `setAttribute()`: Define ou altera um atributo.

```
titulo.setAttribute("class", "destaque");
```

2. `getAttribute()`: Obtém o valor de um atributo.

```
let classe = titulo.getAttribute("class");
```

Alterando Estilos:

1. `style`: Altera estilos CSS diretamente.

```
titulo.style.color = "red";
```

```
titulo.style.fontSize = "24px";
```

4. Adicionando e Removendo Elementos

Adicionando:

1. `createElement()`: Cria um novo elemento.

```
let novoParagrafo = document.createElement("p");
```

```
novoParagrafo.textContent = "Este é um novo parágrafo.";
```

2. `appendChild()`: Adiciona um elemento como filho de outro.

```
document.body.appendChild(novoParagrafo);
```

Removendo:

1. `removeChild()`: Remove um elemento filho.

```
let container = document.getElementById("container");
```

```
container.removeChild(paragrafo);
```

2. `remove()`: Remove o próprio elemento.

```
paragrafo.remove();
```

5. Eventos

O que são Eventos?

Eventos são ações que ocorrem na página, como cliques, movimentos do mouse, pressionamento de teclas, etc. Podemos "ouvir" esses eventos e executar funções em resposta.

Adicionando Eventos:

1. `addEventListener()`: Adiciona um "ouvinte" de evento a um elemento.

```
let botao = document.querySelector("button");
```

```
botao.addEventListener("click", function() {
```

```
 alert("Botão clicado!");
```

```
});
```

Tipos de Eventos Comuns:

- `click`: Clique do mouse.
- `mouseover`: Mouse sobre o elemento.
- `keydown`: Tecla pressionada.
- `submit`: Formulário enviado.

6. Exercícios de Fixação

1. Seleção e Manipulação:

- Crie uma página com um título (`<h1>`) e um parágrafo (`<p>`).
- Use JavaScript para alterar o texto do título e a cor do parágrafo.

## 2. Adicionando Elementos:

- Crie um botão que, ao ser clicado, adiciona um novo parágrafo à página.

## 3. Eventos:

- Crie um botão que, ao ser clicado, exibe um alerta com a mensagem "Olá, Mundo!".

## 4. Removendo Elementos:

- Crie um botão que, ao ser clicado, remove um parágrafo específico da página.

## 5. Desafio:

- Crie uma lista de tarefas onde o usuário pode adicionar novas tarefas digitando em um campo de texto e clicando em um botão.

## 7. Aplicação Prática: Lista de Tarefas Interativa

### Passo a Passo:

1. Crie um arquivo **"lista-tarefas.html"** com o seguinte código:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Lista de Tarefas</title>
 <style>
 .tarefa {
 margin: 10px 0;
 padding: 10px;
 border: 1px solid #ccc;
 border-radius: 5px;
 }
 </style>
</head>
<body>
 <h1>Lista de Tarefas</h1>
 <input type="text" id="novaTarefa" placeholder="Digite uma nova tarefa">
 <button id="adicionarTarefa">Adicionar</button>
 <div id="listaTarefas"></div>

 <script>
 // Seleciona elementos
 const inputTarefa = document.getElementById("novaTarefa");
 const botaoAdicionar = document.getElementById("adicionarTarefa");
 const listaTarefas = document.getElementById("listaTarefas");

 // Função para adicionar tarefa
 botaoAdicionar.addEventListener("click", function() {
 if (inputTarefa.value.trim() !== "") {
 // Cria um novo elemento de tarefa
```

```

const novaTarefa = document.createElement("div");
novaTarefa.className = "tarefa";
novaTarefa.textContent = inputTarefa.value;

// Adiciona a tarefa à lista
listaTarefas.appendChild(novaTarefa);

// Limpa o campo de texto
inputTarefa.value = "";
}
});
</script>
</body>
</html>

```

2. Abra o arquivo no navegador e teste a funcionalidade.

## 8. Respostas dos Exercícios

### 1. Seleção e Manipulação:

```

let titulo = document.querySelector("h1");
titulo.textContent = "Novo Título";

let paragrafo = document.querySelector("p");
paragrafo.style.color = "blue";

```

### 2. Adicionando Elementos:

```

let botao = document.querySelector("button");
botao.addEventListener("click", function() {
 let novoParagrafo = document.createElement("p");
 novoParagrafo.textContent = "Este é um novo parágrafo.";
 document.body.appendChild(novoParagrafo);
});

```

### 3. Eventos:

```

let botao = document.querySelector("button");
botao.addEventListener("click", function() {
 alert("Olá, Mundo!");
});

```

### 4. Removendo Elementos:

```

let botao = document.querySelector("button");
botao.addEventListener("click", function() {
 let paragrafo = document.querySelector("p");
 paragrafo.remove();
});

```

### 5. Desafio:

- Veja o exemplo da **Lista de Tarefas Interativa** acima.

## Capítulo 7:

### Validação de Formulários e Interação com o Usuário

## ### Capítulo 7: Validação de Formulários e Interação com o Usuário

### #### 1. Teoria: Validação de Formulários

O que é Validação?

Validação é o processo de garantir que os dados inseridos pelo usuário em um formulário estejam corretos e atendam a determinados critérios antes de serem enviados.

Tipos de Validação:

#### 1. Validação do Lado do Cliente (Front-end):

- Feita com JavaScript antes de enviar os dados ao servidor.
- Melhora a experiência do usuário, evitando recarregamentos desnecessários.

#### 2. Validação do Lado do Servidor (Back-end):

- Feita no servidor após o envio dos dados.
- Necessária para garantir a segurança e integridade dos dados.

Campos Comuns e Validações:

- Campo de Texto: Verificar se está vazio ou se atende a um padrão (ex: e-mail).
- Senha: Verificar se atende a requisitos de complexidade.
- Número: Verificar se está dentro de um intervalo válido.
- Checkbox/Radio: Verificar se pelo menos uma opção foi selecionada.

### #### 2. Selecionando e Manipulando Formulários

Selecionando Formulários:

#### 1. ``document.querySelector()``: Seleciona o formulário.

```
let formulario = document.querySelector("form");
```

#### 2. ``document.getElementById()``: Seleciona campos específicos.

```
let campoEmail = document.getElementById("email");
```

Manipulando Eventos de Formulários:

#### 1. ``submit``: Evento disparado quando o formulário é enviado.

```
formulario.addEventListener("submit", function(event) {
 event.preventDefault(); // Impede o envio do formulário
 validarFormulario();
});
```

#### 2. ``input``: Evento disparado quando o valor de um campo é alterado.

```
campoEmail.addEventListener("input", function() {
 validarEmail(campoEmail.value);
});
```

### #### 3. Técnicas de Validação

#### Validação Básica:

##### 1. Verificar se um campo está vazio:

```
if (campoEmail.value.trim() === "") {
 alert("O campo e-mail é obrigatório.");
 return false;
}
```

##### 2. Verificar o formato de um e-mail:

```
function validarEmail(email) {
 const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 return regex.test(email);
}
```

##### 3. Verificar o comprimento de uma senha:

```
if (campoSenha.value.length < 8) {
 alert("A senha deve ter pelo menos 8 caracteres.");
 return false;
}
```

#### Exibindo Mensagens de Erro:

- Use elementos HTML para exibir mensagens de erro próximas aos campos.

html

```
<input type="email" id="email">
E-mail inválido.
```

```
function validarEmail(email) {
 const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 if (!regex.test(email)) {
 document.getElementById("erroEmail").style.display = "inline";
 return false;
 } else {
 document.getElementById("erroEmail").style.display = "none";
 return true;
 }
}
```

#### #### 4. Exercícios de Fixação

1. Validação de Campo Obrigatório:

- Crie um formulário com um campo de nome.
- Valide se o campo está preenchido antes de enviar o formulário.

2. Validação de E-mail:

- Adicione um campo de e-mail ao formulário.
- Valide se o e-mail está no formato correto.

3. Validação de Senha:

- Adicione um campo de senha ao formulário.
- Valide se a senha tem pelo menos 8 caracteres.

4. Exibindo Mensagens de Erro:

- Exiba mensagens de erro próximas aos campos quando a validação falhar.

5. Desafio:

- Crie um formulário de cadastro com campos para nome, e-mail, senha e confirmação de senha.
- Valide todos os campos e exiba mensagens de erro personalizadas.

#### #### 5. Aplicação Prática: Formulário de Cadastro

Passo a Passo:

1. Crie um arquivo `cadastro.html` com o seguinte código:

```
html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Formulário de Cadastro</title>
 <style>
 .erro {
 color: red;
 display: none;
 }
 </style>
</head>
<body>
 <h1>Formulário de Cadastro</h1>
 <form id="formCadastro">
 <div>
 <label for="nome">Nome:</label>
 <input type="text" id="nome">
```

```

 O nome é obrigatório.
</div>
<div>
 <label for="email">E-mail:</label>
 <input type="email" id="email">
 E-mail inválido.
</div>
<div>
 <label for="senha">Senha:</label>
 <input type="password" id="senha">
 A senha deve ter pelo menos 8 caracteres.
</div>
<button type="submit">Cadastrar</button>
</form>

<script>
 const formulario = document.getElementById("formCadastro");
 const campoNome = document.getElementById("nome");
 const campoEmail = document.getElementById("email");
 const campoSenha = document.getElementById("senha");

 formulario.addEventListener("submit", function(event) {
 event.preventDefault(); // Impede o envio do formulário

 // Validação do nome
 if (campoNome.value.trim() === "") {
 document.getElementById("erroNome").style.display = "inline";
 } else {
 document.getElementById("erroNome").style.display = "none";
 }

 // Validação do e-mail
 if (!validarEmail(campoEmail.value)) {
 document.getElementById("erroEmail").style.display = "inline";
 } else {
 document.getElementById("erroEmail").style.display = "none";
 }

 // Validação da senha
 if (campoSenha.value.length < 8) {
 document.getElementById("erroSenha").style.display = "inline";
 } else {
 document.getElementById("erroSenha").style.display = "none";
 }
 });

 function validarEmail(email) {
 const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 return regex.test(email);
 }
</script>

```

```
</body>
</html>
```

2. Abra o arquivo no navegador e teste a validação do formulário.

#### #### 6. Respostas dos Exercícios

##### 1. Validação de Campo Obrigatório:

```
if (campoNome.value.trim() === "") {
 alert("O campo nome é obrigatório.");
 return false;
}
```

##### 2. Validação de E-mail:

```
function validarEmail(email) {
 const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 return regex.test(email);
}
```

##### 3. Validação de Senha:

```
if (campoSenha.value.length < 8) {
 alert("A senha deve ter pelo menos 8 caracteres.");
 return false;
}
```

##### 4. Exibindo Mensagens de Erro:

```
if (campoNome.value.trim() === "") {
 document.getElementById("erroNome").style.display = "inline";
} else {
 document.getElementById("erroNome").style.display = "none";
}
```

##### 5. Desafio:

- Veja o exemplo do Formulário de Cadastro acima.

## Capítulo 8:

Armazenamento no Navegador (LocalStorage e SessionStorage)

## 1. Teoria: Armazenamento no Navegador

### O que é Armazenamento no Navegador?

O armazenamento no navegador permite salvar dados diretamente no computador do usuário. Isso é útil para manter informações como preferências, dados de sessão, ou até mesmo conteúdo de aplicações web.

### Tipos de Armazenamento:

1. **localStorage**: Armazena dados sem data de expiração. Os dados persistem mesmo após fechar o navegador.
2. **sessionStorage**: Armazena dados apenas durante a sessão atual. Os dados são apagados ao fechar o navegador.

### Diferenças:

- **localStorage**: é ideal para dados que precisam ser mantidos por longos períodos.
- **sessionStorage**: é útil para dados temporários, como informações de uma sessão de usuário.

## 2. Usando **localStorage**

### Métodos Principais:

1. **setItem()**: Salva um item no `localStorage`.  
`localStorage.setItem("nome", "João");`
2. **getItem()**: Recupera um item do `localStorage`.  
`let nome = localStorage.getItem("nome");`  
`console.log(nome); // "João"`
3. **removeItem()**: Remove um item do `localStorage`.  
`localStorage.removeItem("nome");`
4. **clear()**: Limpa todos os itens do `localStorage`.  
`localStorage.clear();`

### Exemplo:

```
// Salvando dados
localStorage.setItem("usuario", JSON.stringify({ nome: "João", idade: 25 }));

// Recuperando dados
let usuario = JSON.parse(localStorage.getItem("usuario"));
console.log(usuario); // { nome: "João", idade: 25 }
```

## 3. Usando **sessionStorage**

### Métodos Principais:

- Os métodos são os mesmos do `localStorage`:
  1. **setItem()**: Salva um item no `sessionStorage`.  
`sessionStorage.setItem("token", "abc123");`
  2. **getItem()**: Recupera um item do `sessionStorage`.

```
let token = sessionStorage.getItem("token");
console.log(token); // "abc123"
```

3. `removeItem()`: Remove um item do `sessionStorage`.

```
sessionStorage.removeItem("token");
```

4. `clear()`: Limpa todos os itens do `sessionStorage`.

```
sessionStorage.clear();
```

Exemplo:

```
// Salvando dados
sessionStorage.setItem("config", JSON.stringify({ tema: "escuro", idioma: "pt-BR" }));
```

```
// Recuperando dados
let config = JSON.parse(sessionStorage.getItem("config"));
console.log(config); // { tema: "escuro", idioma: "pt-BR" }
```

## 4. Exercícios de Fixação

### 1. Salvando e Recuperando Dados:

- Crie um formulário simples com um campo de texto.
- Salve o valor digitado no `localStorage` e exiba-o ao recarregar a página.

### 2. Tema Escuro/Claro:

- Crie um botão que alterna entre temas claro e escuro.
- Salve a preferência do tema no `localStorage` e aplique-a ao recarregar a página.

### 3. Lista de Tarefas com `localStorage`:

- Modifique a lista de tarefas do Capítulo 6 para salvar as tarefas no `localStorage`.
- Recupere as tarefas ao carregar a página.

### 4. Desafio:

- Crie um sistema de login simples que salva o nome do usuário no `sessionStorage`.
- Exiba uma mensagem de boas-vindas ao usuário após o login.

## 5. Aplicação Prática: Lista de Tarefas com `localStorage`

Passo a Passo:

1. Crie um arquivo `lista-tarefas-localstorage.html` com o seguinte código:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Lista de Tarefas com LocalStorage</title>
 <style>
 .tarefa {
 margin: 10px 0;
```

```

padding: 10px;
border: 1px solid #ccc;
border-radius: 5px;
}
</style>
</head>
<body>
<h1>Lista de Tarefas</h1>
<input type="text" id="novaTarefa" placeholder="Digite uma nova tarefa">
<button id="adicionarTarefa">Adicionar</button>
<div id="listaTarefas"></div>

<script>
// Seleciona elementos
const inputTarefa = document.getElementById("novaTarefa");
const botaoAdicionar = document.getElementById("adicionarTarefa");
const listaTarefas = document.getElementById("listaTarefas");

// Carrega tarefas salvas ao iniciar
let tarefas = JSON.parse(localStorage.getItem("tarefas")) || [];

function renderizarTarefas() {
 listaTarefas.innerHTML = "";
 tarefas.forEach((tarefa, index) => {
 const novaTarefa = document.createElement("div");
 novaTarefa.className = "tarefa";
 novaTarefa.textContent = tarefa;
 listaTarefas.appendChild(novaTarefa);
 });
}

// Adiciona nova tarefa
botaoAdicionar.addEventListener("click", function() {
 if (inputTarefa.value.trim() !== "") {
 tarefas.push(inputTarefa.value);
 localStorage.setItem("tarefas", JSON.stringify(tarefas));
 inputTarefa.value = "";
 renderizarTarefas();
 }
});

// Renderiza tarefas ao carregar a página
renderizarTarefas();
</script>
</body>
</html>

```

2. Abra o arquivo no navegador e teste a funcionalidade. As tarefas serão salvas no `localStorage` e persistirão após recarregar a página.

## 6. Respostas dos Exercícios

### 1. Salvando e Recuperando Dados:

```
<input type="text" id="texto">
<button id="salvar">Salvar</button>
<p id="resultado"></p>

<script>
 const campoTexto = document.getElementById("texto");
 const botaoSalvar = document.getElementById("salvar");
 const resultado = document.getElementById("resultado");

 // Recupera o valor salvo ao carregar a página
 campoTexto.value = localStorage.getItem("textoSalvo") || "";

 botaoSalvar.addEventListener("click", function() {
 localStorage.setItem("textoSalvo", campoTexto.value);
 resultado.textContent = "Texto salvo: " + campoTexto.value;
 });
</script>
```

### 2. Tema Escuro/Claro:

```
<button id="alternarTema">Alternar Tema</button>

<script>
 const botaoTema = document.getElementById("alternarTema");
 const temaSalvo = localStorage.getItem("tema");

 if (temaSalvo === "escuro") {
 document.body.style.backgroundColor = "#333";
 document.body.style.color = "#fff";
 }

 botaoTema.addEventListener("click", function() {
 if (document.body.style.backgroundColor === "white" || !document.body.style.backgroundColor) {
 document.body.style.backgroundColor = "#333";
 document.body.style.color = "#fff";
 localStorage.setItem("tema", "escuro");
 } else {
 document.body.style.backgroundColor = "white";
 document.body.style.color = "#000";
 localStorage.setItem("tema", "claro");
 }
 });
</script>
```

### 3. Lista de Tarefas com `localStorage`:

- Veja o exemplo da Lista de Tarefas com `localStorage` acima.

#### 4. Desafio:

```
<input type="text" id="nomeUsuario" placeholder="Digite seu nome">
<button id="login">Login</button>
<p id="mensagem"></p>

<script>
 const campoNome = document.getElementById("nomeUsuario");
 const botaoLogin = document.getElementById("login");
 const mensagem = document.getElementById("mensagem");

 // Verifica se há um usuário logado ao carregar a página
 const usuario = sessionStorage.getItem("usuario");
 if (usuario) {
 mensagem.textContent = "Bem-vindo de volta, " + usuario + "!";
 }

 botaoLogin.addEventListener("click", function() {
 if (campoNome.value.trim() !== "") {
 sessionStorage.setItem("usuario", campoNome.value);
 mensagem.textContent = "Bem-vindo, " + campoNome.value + "!";
 }
 });
</script>
```

## Capítulo 9:

### Introdução ao AJAX e Fetch API

## 1. Teoria: O que é AJAX?

### Definição:

AJAX (Asynchronous JavaScript and XML) é uma técnica que permite fazer requisições assíncronas a um servidor sem recarregar a página. Isso possibilita atualizar partes específicas de uma página web, melhorando a experiência do usuário.

### Como Funciona:

1. O JavaScript envia uma requisição ao servidor.
2. O servidor processa a requisição e envia uma resposta.
3. O JavaScript recebe a resposta e atualiza a página.

### Benefícios:

- Melhora a performance e a usabilidade da aplicação.
- Permite carregar dados dinamicamente, sem recarregar a página.

## 2. Teoria: Fetch API

### O que é Fetch API?

A Fetch API é uma interface moderna para fazer requisições HTTP em JavaScript. Ela substitui o antigo `XMLHttpRequest` e é mais poderosa e fácil de usar.

### Sintaxe Básica:

```
fetch(url)
 .then(response => response.json())
 .then(data => console.log(data))
 .catch(error => console.error("Erro:", error));
```

### Métodos HTTP Comuns:

- GET: Solicita dados do servidor.
- POST: Envia dados ao servidor.
- PUT: Atualiza dados no servidor.
- DELETE: Remove dados do servidor.

### Exemplo de Requisição GET:

```
fetch("https://jsonplaceholder.typicode.com/posts")
 .then(response => response.json())
 .then(posts => console.log(posts))
 .catch(error => console.error("Erro:", error));
```

### Exemplo de Requisição POST:

```
fetch("https://jsonplaceholder.typicode.com/posts", {
 method: "POST",
 headers: {
 "Content-Type": "application/json"
 },
 body: JSON.stringify({
 title: "Novo Post",
 body: "Conteúdo do post",
 })
})
```

```

 userId: 1
 })
 })
 .then(response => response.json())
 .then(data => console.log(data))
 .catch(error => console.error("Erro:", error));

```

### 3. Trabalhando com JSON

#### O que é JSON?

JSON (JavaScript Object Notation) é um formato leve para troca de dados. Ele é fácil de ler e escrever para humanos e fácil de parsear e gerar para máquinas.

#### Exemplo de JSON:

```

json
{
 "nome": "João",
 "idade": 25,
 "isAdmin": false
}

```

#### Convertendo JSON:

##### 1. JSON.stringify(): Converte um objeto JavaScript em uma string JSON.

```

let usuario = { nome: "João", idade: 25 };
let json = JSON.stringify(usuario);
console.log(json); // '{"nome":"João","idade":25}'

```

##### 2. JSON.parse(): Converte uma string JSON em um objeto JavaScript.

```

let json = '{"nome":"João","idade":25}';
let usuario = JSON.parse(json);
console.log(usuario.nome); // "João"

```

### 4. Exercícios de Fixação

#### 1. Requisição GET:

- Use a Fetch API para buscar dados de uma API pública (ex: [JSONPlaceholder](https://jsonplaceholder.typicode.com/)).
- Exiba os dados no console.

#### 2. Requisição POST:

- Crie um formulário simples com campos para título e conteúdo.
- Use a Fetch API para enviar os dados do formulário para uma API.

#### 3. Manipulando Respostas:

- Após receber os dados de uma requisição GET, exiba-os em uma lista na página.

#### 4. Desafio:

- Crie uma aplicação que busca e exibe dados de uma API de previsão do tempo (ex: [OpenWeatherMap](https://openweathermap.org/)).

## 5. Aplicação Prática: Lista de Posts com Fetch API

Passo a Passo:

1. Crie um arquivo `lista-posts.html` com o seguinte código:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Lista de Posts</title>
 <style>
 .post {
 margin: 10px 0;
 padding: 10px;
 border: 1px solid #ccc;
 border-radius: 5px;
 }
 </style>
</head>
<body>
 <h1>Lista de Posts</h1>
 <div id="listaPosts"></div>

 <script>
 const listaPosts = document.getElementById("listaPosts");

 // Busca posts da API
 fetch("https://jsonplaceholder.typicode.com/posts")
 .then(response => response.json())
 .then(posts => {
 posts.forEach(post => {
 const divPost = document.createElement("div");
 divPost.className = "post";
 divPost.innerHTML = `
 <h2>${post.title}</h2>
 <p>${post.body}</p>
 `;
 listaPosts.appendChild(divPost);
 });
 })
 .catch(error => console.error("Erro:", error));
 </script>
</body>
</html>
```

2. Abra o arquivo no navegador e veja a lista de posts carregada dinamicamente.

## 6. Respostas dos Exercícios

1. Requisição GET:

```
fetch("https://jsonplaceholder.typicode.com/posts")
 .then(response => response.json())
 .then(posts => console.log(posts))
 .catch(error => console.error("Erro:", error));
```

## 2. Requisição POST:

```
<form id="formPost">
 <input type="text" id="titulo" placeholder="Título">
 <textarea id="conteudo" placeholder="Conteúdo"></textarea>
 <button type="submit">Enviar</button>
</form>

<script>
 const formPost = document.getElementById("formPost");

 formPost.addEventListener("submit", function(event) {
 event.preventDefault();

 const titulo = document.getElementById("titulo").value;
 const conteudo = document.getElementById("conteudo").value;

 fetch("https://jsonplaceholder.typicode.com/posts", {
 method: "POST",
 headers: {
 "Content-Type": "application/json"
 },
 body: JSON.stringify({
 title: titulo,
 body: conteudo,
 userId: 1
 })
 })
 .then(response => response.json())
 .then(data => console.log(data))
 .catch(error => console.error("Erro:", error));
 });
</script>
```

## 3. Manipulando Respostas:

```
fetch("https://jsonplaceholder.typicode.com/posts")
 .then(response => response.json())
 .then(posts => {
 const lista = document.createElement("ul");
 posts.forEach(post => {
 const item = document.createElement("li");
 item.textContent = post.title;
```

```
 lista.appendChild(item);
 });
 document.body.appendChild(lista);
})
.catch(error => console.error("Erro:", error));
```

#### 4. Desafio:

- Use a API do OpenWeatherMap para buscar a previsão do tempo de uma cidade e exiba os dados na página.

## Capítulo 10:

### Programação Orientada a Objetos (POO) em JavaScript

## 1. Teoria: O que é POO?

### Definição:

Programação Orientada a Objetos (POO) é um paradigma de programação que organiza o código em "objetos", que são instâncias de "classes". Esses objetos encapsulam dados (propriedades) e comportamentos (métodos).

### Conceitos Fundamentais:

1. Classe: Um modelo ou blueprint para criar objetos.
2. Objeto: Uma instância de uma classe.
3. Propriedades: Variáveis que pertencem a um objeto.
4. Métodos: Funções que pertencem a um objeto.
5. Herança: Capacidade de uma classe herdar propriedades e métodos de outra classe.
6. Encapsulamento: Ocultar detalhes internos e expor apenas o necessário.
7. Polimorfismo: Capacidade de um objeto se comportar de múltiplas formas.

## 2. Criando Classes e Objetos

### Sintaxe de Classe:

```
class Pessoa {
 constructor(nome, idade) {
 this.nome = nome;
 this.idade = idade;
 }

 saudacao() {
 return `Olá, meu nome é ${this.nome} e eu tenho ${this.idade} anos.`;
 }
}
```

### Criando Objetos:

```
const pessoa1 = new Pessoa("João", 25);
console.log(pessoa1.saudacao()); // "Olá, meu nome é João e eu tenho 25 anos."
```

## 3. Herança

### Sintaxe de Herança:

```
class Estudante extends Pessoa {
 constructor(nome, idade, curso) {
 super(nome, idade); // Chama o construtor da classe pai
 this.curso = curso;
 }

 estudar() {
 return `${this.nome} está estudando ${this.curso}.`;
 }
}
```

Usando a Classe Filha:

```
const estudante1 = new Estudante("Maria", 22, "JavaScript");
console.log(estudante1.saudacao()); // "Olá, meu nome é Maria e eu tenho 22 anos."
console.log(estudante1.estudar()); // "Maria está estudando JavaScript."
```

#### 4. Métodos Estáticos

O que são Métodos Estáticos?

Métodos estáticos pertencem à classe, não às instâncias. Eles são chamados diretamente na classe, sem a necessidade de criar um objeto.

Sintaxe:

```
class Matematica {
 static soma(a, b) {
 return a + b;
 }
}
```

Usando Métodos Estáticos:

```
console.log(Matematica.soma(2, 3)); // 5
```

#### 5. Exercícios de Fixação

##### 1. Criando Classes:

- Crie uma classe `Carro` com propriedades `marca`, `modelo` e `ano`.
- Adicione um método `detalhes` que retorna uma string com as informações do carro.

##### 2. Herança:

- Crie uma classe `Caminhao` que herda de `Carro` e adiciona a propriedade `capacidadeCarga`.

##### 3. Métodos Estáticos:

- Adicione um método estático `info` na classe `Carro` que retorna uma mensagem sobre carros.

##### 4. Desafio:

- Crie uma classe `ContaBancaria` com propriedades `titular` e `saldo`.
- Adicione métodos `depositar` e `sacar`.
- Crie uma classe `ContaPoupanca` que herda de `ContaBancaria` e adiciona a propriedade `taxaJuros`.

#### 6. Aplicação Prática: Sistema de Gerenciamento de Produtos

Passo a Passo:

1. Crie um arquivo `produtos.html` com o seguinte código:

```
<!DOCTYPE html>
```

```

<html lang="pt-BR">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Gerenciamento de Produtos</title>
</head>
<body>
 <h1>Gerenciamento de Produtos</h1>
 <script>
 class Produto {
 constructor(nome, preco) {
 this.nome = nome;
 this.preco = preco;
 }

 detalhes() {
 return ` ${this.nome} - R$ ${this.preco.toFixed(2)} `;
 }
 }

 class ProdutoDesconto extends Produto {
 constructor(nome, preco, desconto) {
 super(nome, preco);
 this.desconto = desconto;
 }

 precoComDesconto() {
 return this.preco * (1 - this.desconto);
 }

 detalhes() {
 return ` ${super.detalhes()} (${this.desconto * 100}% de desconto) `;
 }
 }

 const produto1 = new Produto("Notebook", 3000);
 const produto2 = new ProdutoDesconto("Smartphone", 1500, 0.1);

 console.log(produto1.detalhes()); // "Notebook - R$ 3000.00"
 console.log(produto2.detalhes()); // "Smartphone - R$ 1500.00 (10% de desconto)"
 console.log("Preço com desconto:", produto2.precoComDesconto()); // 1350
 </script>
</body>
</html>

```

2. Abra o arquivo no navegador e veja os detalhes dos produtos no console.

## 7. Respostas dos Exercícios

### 1. Criando Classes:

```

class Carro {
 constructor(marca, modelo, ano) {
 this.marca = marca;
 this.modelo = modelo;
 this.ano = ano;
 }

 detalhes() {
 return `${this.marca} ${this.modelo} (${this.ano})`;
 }
}

const carro1 = new Carro("Toyota", "Corolla", 2020);
console.log(carro1.detalhes()); // "Toyota Corolla (2020)"

```

## 2. Herança:

```

class Caminhao extends Carro {
 constructor(marca, modelo, ano, capacidadeCarga) {
 super(marca, modelo, ano);
 this.capacidadeCarga = capacidadeCarga;
 }

 detalhes() {
 return `${super.detalhes()} - Carga: ${this.capacidadeCarga} toneladas`;
 }
}

const caminhao1 = new Caminhao("Volvo", "FH", 2019, 15);
console.log(caminhao1.detalhes()); // "Volvo FH (2019) - Carga: 15 toneladas"

```

## 3. Métodos Estáticos:

```

class Carro {
 constructor(marca, modelo, ano) {
 this.marca = marca;
 this.modelo = modelo;
 this.ano = ano;
 }

 detalhes() {
 return `${this.marca} ${this.modelo} (${this.ano})`;
 }

 static info() {
 return "Carros são veículos automotores.";
 }
}

```

```
console.log(Carro.info()); // "Carros são veículos automotores."
```

#### 4. Desafio:

```
class ContaBancaria {
 constructor(titular, saldo) {
 this.titular = titular;
 this.saldo = saldo;
 }

 depositar(valor) {
 this.saldo += valor;
 }

 sacar(valor) {
 if (valor > this.saldo) {
 return "Saldo insuficiente.";
 }
 this.saldo -= valor;
 }
}

class ContaPoupanca extends ContaBancaria {
 constructor(titular, saldo, taxaJuros) {
 super(titular, saldo);
 this.taxaJuros = taxaJuros;
 }

 aplicarJuros() {
 this.saldo *= (1 + this.taxaJuros);
 }
}

const conta1 = new ContaPoupanca("João", 1000, 0.05);
conta1.aplicarJuros();
console.log(conta1.saldo); // 1050
```

## Capítulo 11: Introdução a Frameworks (React, Vue ou Angular)

## 1. Teoria: O que são Frameworks?

### Definição:

Frameworks são estruturas de software que fornecem uma base para o desenvolvimento de aplicações. Eles incluem bibliotecas, ferramentas e padrões que ajudam a organizar o código e acelerar o desenvolvimento.

### Por que usar Frameworks?

- **Produtividade:** Facilita a criação de aplicações complexas.
- **Organização:** Fornece uma estrutura clara para o código.
- **Reutilização:** Componentes e funcionalidades podem ser reutilizados.
- **Comunidade:** Amplo suporte e recursos disponíveis.

### Frameworks Populares para JavaScript:

1. **React:** Desenvolvido pelo Facebook, focado em componentes reutilizáveis.
2. **Vue:** Framework progressivo, fácil de integrar em projetos existentes.
3. **Angular:** Desenvolvido pelo Google, completo e robusto.

## 2. Introdução ao React

### O que é React?

React é uma biblioteca JavaScript para construir interfaces de usuário (UI) com componentes reutilizáveis. Ele usa uma sintaxe chamada JSX, que permite escrever HTML diretamente no JavaScript.

### Conceitos Básicos:

1. **Componentes:** Blocos de construção da UI.
2. **JSX:** Sintaxe que combina HTML e JavaScript.
3. **Estado (State):** Dados que podem mudar e afetar a renderização.
4. **Props:** Propriedades passadas para componentes.

### Exemplo de Componente React:

```
function Saudacao(props) {
 return <h1>Olá, {props.nome}</h1>;
}
```

```
ReactDOM.render(<Saudacao nome="João" />, document.getElementById("root"));
```

### 3. Introdução ao Vue

O que é Vue?

Vue é um framework progressivo para construir interfaces de usuário. Ele é conhecido por sua simplicidade e flexibilidade, permitindo que você comece pequeno e cresça conforme necessário.

Conceitos Básicos:

1. Instância Vue: O núcleo de uma aplicação Vue.
2. Diretivas: Comandos especiais no HTML (ex: `v-bind`, `v-model`).
3. Componentes: Blocos de construção reutilizáveis.
4. Reatividade: Atualizações automáticas da UI quando os dados mudam.

Exemplo de Instância Vue:

```
<div id="app">
 <p>{{ mensagem }}</p>
</div>
<script>
 const app = new Vue({
 el: "#app",
 data: {
 mensagem: "Olá, Vue!"
 }
 });
</script>
```

### 4. Introdução ao Angular

O que é Angular?

Angular é um framework completo para construir aplicações web. Ele inclui tudo o que você precisa, desde roteamento até gerenciamento de estado.

Conceitos Básicos:

1. Componentes: Blocos de construção da UI.

2. Módulos: Organização do código em funcionalidades.
3. Serviços: Lógica reutilizável.
4. Two-Way Data Binding: Atualizações automáticas entre a UI e os dados.

Exemplo de Componente Angular:

```
import { Component } from "@angular/core";

@Component({
 selector: "app-saudacao",
 template: `<h1>Olá, {{ nome }}!</h1>`
})
export class SaudacaoComponent {
 nome = "João";
}
```

## 5. Exercícios de Fixação

### 1. React:

- Crie um componente `Contador` que exibe um número e um botão para incrementá-lo.

### 2. Vue:

- Crie uma instância Vue que exibe uma lista de tarefas e permite adicionar novas tarefas.

### 3. Angular:

- Crie um componente `Saudacao` que exibe uma mensagem personalizada com base em uma entrada do usuário.

### 4. Desafio:

- Escolha um framework (React, Vue ou Angular) e crie uma aplicação simples de lista de tarefas.

## 6. Aplicação Prática: Lista de Tarefas com React

Passo a Passo:

### 1. Crie um arquivo `lista-tarefas-react.html` com o seguinte código:

```
<!DOCTYPE html>

<html lang="pt-BR">

<head>
```

```
<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Lista de Tarefas com React</title>

<script src="https://unpkg.com/react@17/umd/react.development.js"></script>

<script src="https://unpkg.com/react-dom@17/umd/react-dom.development.js"></script>

<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

</head>

<body>

 <div id="root"></div>

 <script type="text/babel">

 function ListaTarefas() {

 const [tarefas, setTarefas] = React.useState([]);

 const [novaTarefa, setNovaTarefa] = React.useState("");

 const adicionarTarefa = () => {

 if (novaTarefa.trim() !== "") {

 setTarefas([...tarefas, novaTarefa]);

 setNovaTarefa("");

 }

 };

 return (

 <div>

 <h1>Lista de Tarefas</h1>

 <input

 type="text"

 value={novaTarefa}

 onChange={(e) => setNovaTarefa(e.target.value)}

 placeholder="Digite uma nova tarefa"

 />

 <button onClick={adicionarTarefa}>Adicionar</button>


```

```

 {tarefas.map((tarefa, index) => (
 <li key={index}>{tarefa}
))}

</div>

);
}

```

```

ReactDOM.render(<ListaTarefas />, document.getElementById("root"));
</script>
</body>
</html>

```

2. Abra o arquivo no navegador e teste a funcionalidade.

## 7. Respostas dos Exercícios

### 1. React:

```

function Contador() {
 const [contador, setContador] = React.useState(0);
 return (
 <div>
 <p>{contador}</p>
 <button onClick={() => setContador(contador + 1)}>Incrementar</button>
 </div>
);
}

ReactDOM.render(<Contador />, document.getElementById("root"));

```

### 2. Vue:

```

<div id="app">


```

```

 <li v-for="tarefa in tarefas">{{ tarefa }}

<input v-model="novaTarefa" placeholder="Digite uma nova tarefa">

<button @click="adicionarTarefa">Adicionar</button>

</div>

<script>

const app = new Vue({

 el: "#app",

 data: {

 tarefas: [],

 novaTarefa: ""

 },

 methods: {

 adicionarTarefa() {

 if (this.novaTarefa.trim() !== "") {

 this.tarefas.push(this.novaTarefa);

 this.novaTarefa = "";

 }

 }

 }

});

</script>

```

### 3. Angular:

```

import { Component } from "@angular/core";

@Component({

 selector: "app-saudacao",

 template: `

 <input [(ngModel)]="nome" placeholder="Digite seu nome">

 <h1>Olá, {{ nome }}!</h1>

```

```
 ,
```

```
 })
```

```
export class SaudacaoComponent {
 nome = "";
}
```

#### 4. Desafio:

- Veja o exemplo da [Lista de Tarefas com React](#) acima.

## Capítulo 12: Projeto Final

## 1. Objetivo do Projeto Final

O objetivo deste capítulo é consolidar todo o conhecimento adquirido ao longo dos capítulos anteriores, aplicando-o na criação de uma aplicação web completa. O projeto final será uma aplicação de gerenciamento de tarefas (To-Do List) com funcionalidades avançadas, como:

- Adicionar, editar e remover tarefas.
- Marcar tarefas como concluídas.
- Filtrar tarefas por status (concluídas/pendentes).
- Salvar tarefas no `localStorage` para persistência de dados.
- Utilizar um framework (React, Vue ou Angular) para estruturar a aplicação.

## 2. Escopo do Projeto

Funcionalidades:

1. Adicionar Tarefa:
  - Um campo de texto e um botão para adicionar novas tarefas.
2. Listar Tarefas:
  - Exibir todas as tarefas em uma lista.
  - Cada tarefa deve ter um checkbox para marcar como concluída.
3. Editar Tarefa:
  - Permitir editar o texto de uma tarefa existente.
4. Remover Tarefa:
  - Um botão para remover uma tarefa da lista.
5. Filtrar Tarefas:
  - Botões para filtrar tarefas por "Todas", "Concluídas" ou "Pendentes".
6. Persistência de Dados:
  - Salvar as tarefas no `localStorage` para que persistam após recarregar a página.

Tecnologias Utilizadas:

- HTML, CSS e JavaScript.
- Framework escolhido (React, Vue ou Angular).
- **localStorage** para armazenamento de dados.

## 3. Estrutura do Projeto

Estrutura de Pastas:

/projeto-final

  /css

    styles.css

  /js

    app.js

  index.html

Estrutura HTML Básica:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
 <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>To-Do List</title>
<link rel="stylesheet" href="css/styles.css">
</head>
<body>
 <div id="app">
 <h1>To-Do List</h1>
 <input type="text" id="novaTarefa" placeholder="Digite uma nova tarefa">
 <button id="adicionarTarefa">Adicionar</button>
 <div id="filtros">
 <button class="filtro" data-filtro="todas">Todas</button>
 <button class="filtro" data-filtro="concluidas">Concluídas</button>
 <button class="filtro" data-filtro="pendentes">Pendentes</button>
 </div>
 <ul id="listaTarefas">
 </div>
 <script src="js/app.js"></script>
</body>
</html>
```

## 4. Implementação do Projeto

Passo a Passo:

### 1. Configuração Inicial:

- Crie a estrutura de pastas e arquivos.
- Adicione o HTML básico e os links para CSS e JavaScript.

### 2. Adicionar Tarefas:

- Capture o valor do campo de texto e adicione uma nova tarefa à lista.
- Atualize a exibição da lista e salve as tarefas no `localStorage`.

### 3. Listar Tarefas:

- Crie uma função para renderizar a lista de tarefas.
- Adicione um checkbox para marcar tarefas como concluídas.

### 4. Editar Tarefas:

- Adicione um botão de edição que permite alterar o texto da tarefa.
- Atualize a lista e o `localStorage` após a edição.

### 5. Remover Tarefas:

- Adicione um botão de remoção para cada tarefa.
- Atualize a lista e o `localStorage` após a remoção.

### 6. Filtrar Tarefas:

- Implemente a lógica para filtrar tarefas por status (concluídas/pendentes).

### 7. Persistência de Dados:

- Use `localStorage` para salvar e carregar as tarefas ao iniciar a aplicação.

## 5. Exemplo de Código

JavaScript (app.js):

```
document.addEventListener("DOMContentLoaded", function() {
 const listaTarefas = document.getElementById("listaTarefas");
 const novaTarefaInput = document.getElementById("novaTarefa");
 const adicionarTarefaBtn = document.getElementById("adicionarTarefa");
 const filtros = document.querySelectorAll(".filtro");
```

```
let tarefas = JSON.parse(localStorage.getItem("tarefas")) || [];
```

```
function renderizarTarefas(filtro = "todas") {
 listaTarefas.innerHTML = "";
 const tarefasFiltradas = tarefas.filter(tarefa => {
 if (filtro === "concluidas") return tarefa.concluida;
 if (filtro === "pendentes") return !tarefa.concluida;
 return true;
 });
```

```
 tarefasFiltradas.forEach((tarefa, index) => {
 const li = document.createElement("li");
 li.innerHTML = `
 <input type="checkbox" ${tarefa.concluida ? "checked" : ""}>
 ${tarefa.texto}
 <button class="editar">Editar</button>
 <button class="remover">Remover</button>
 `;
 listaTarefas.appendChild(li);
```

```
 const checkbox = li.querySelector("input");
 checkbox.addEventListener("change", () => {
 tarefas[index].concluida = checkbox.checked;
 localStorage.setItem("tarefas", JSON.stringify(tarefas));
 renderizarTarefas();
 });
```

```
 const botaoEditar = li.querySelector(".editar");
 botaoEditar.addEventListener("click", () => {
 const novoTexto = prompt("Editar tarefa:", tarefa.texto);
 if (novoTexto !== null) {
 tarefas[index].texto = novoTexto;
 localStorage.setItem("tarefas", JSON.stringify(tarefas));
 renderizarTarefas();
 }
 });
```

```
 const botaoRemover = li.querySelector(".remover");
 botaoRemover.addEventListener("click", () => {
 tarefas.splice(index, 1);
 localStorage.setItem("tarefas", JSON.stringify(tarefas));
 renderizarTarefas();
```

```

 });
 });
}

adicionarTarefaBtn.addEventListener("click", () => {
 const texto = novaTarefaInput.value.trim();
 if (texto !== "") {
 tarefas.push({ texto, concluida: false });
 localStorage.setItem("tarefas", JSON.stringify(tarefas));
 novaTarefaInput.value = "";
 renderizarTarefas();
 }
});

filtros.forEach(filtro => {
 filtro.addEventListener("click", () => {
 const filtroSelecionado = filtro.getAttribute("data-filtro");
 renderizarTarefas(filtroSelecionado);
 });
});

renderizarTarefas();
});

```

## 6. Desafios Adicionais

### 1. Melhorar a Interface:

- Adicione estilos CSS para melhorar a aparência da aplicação.
- Use bibliotecas como Bootstrap ou Materialize para componentes prontos.

### 2. Adicionar Data de Vencimento:

- Permita que o usuário defina uma data de vencimento para cada tarefa.
- Exiba tarefas próximas do vencimento em destaque.

### 3. Ordenação de Tarefas:

- Adicione a funcionalidade de ordenar tarefas por data de criação ou vencimento.

### 4. Integração com Backend:

- Substitua o `localStorage` por uma API backend para persistência de dados.

## 7. Conclusão

Este projeto final consolidou todos os conceitos aprendidos ao longo dos capítulos, desde o básico de JavaScript até a manipulação do DOM, armazenamento no navegador e uso de frameworks. Agora você está pronto para criar suas próprias aplicações web!